

## 1) Arquitectura actual identificada

Conclusión: la aplicación implementa una arquitectura monolítica web basada en Laravel, con estilo MVC + capas de soporte (middleware, políticas/gates, modelos/ORM, vistas Blade) y enfoque multi-tenant por contexto.

| Dimension                  | Estado actual                                                                                              | Nivel        |
|----------------------------|------------------------------------------------------------------------------------------------------------|--------------|
| Tipo de despliegue         | Monolito Laravel en contenedores (PHP-FPM + Nginx + MySQL + Vite para desarrollo).                         | Adecuado     |
| Arquitectura de aplicación | MVC (Controllers, Models Eloquent, Views Blade), con middleware para autenticación, rol y contexto tenant. | Adecuado     |
| Multi-tenancy              | Aislamiento lógico por tenant mediante middleware + trait con global scope + helper de storage tenant.     | Fuerte       |
| Capa de dominio/servicios  | La lógica vive mayormente en controllers y modelos; carpeta Repositories existe pero vacía.                | A<br>mejorar |

## 2) Patrones de diseño detectados

Patrones presentes

- **MVC:** rutas -> controladores -> modelos/vistas, tipico en Laravel.
- **Front Controller:** enrutamiento centralizado por framework.
- **Middleware Chain (Pipeline):** autenticacion, validacion de rol y seteo de tenant en cadena.
- **Active Record (Eloquent):** modelos con relaciones y consultas embebidas.
- **Global Scope:** trait para filtrar datos por tenant de forma transversal.
- **Strategy via Driver Configuration:** cache/queue/session por drivers configurables.
- **View Composer:** inyeccion transversal de configuracion de UI en vistas.
- **RBAC basico:** roles y gates para control de autorizacion.

### Patrones parcialmente aplicados o ausentes

- **Repository Pattern:** no implementado aun (directorio preparado).
- **Service Layer / Use Cases:** no estandarizado; hay casos con alta logica en controller.
- **CQRS / Event-driven:** no evidencias claras en el estado actual.
- **Hexagonal/Clean Architecture:** no aplicado formalmente.

## 3) Diagnostico de calidad arquitectonica

| Criterio                        | Evaluacion | Observacion                                                                 |
|---------------------------------|------------|-----------------------------------------------------------------------------|
| Separacion de responsabilidades | Media      | Base MVC correcta, pero controladores con mucha carga en algunos modulos.   |
| Escalabilidad de codigo         | Media      | Falta capa de servicios/repositorios para crecer sin duplicidad.            |
| Consistencia multi-tenant       | Alta       | Buena estandarizacion de tenant en query y storage.                         |
| Mantenibilidad                  | Media      | El crecimiento funcional podria aumentar deuda tecnica si no se modulariza. |

## 4) Recomendaciones (priorizadas)

### Corto plazo (2 a 4 semanas)

- Definir una capa de **Application Services** para casos de uso complejos (ejemplo: generacion de reportes, carga masiva, reglas de negocio).
- Extraer consultas complejas repetidas a **Query Objects** o Repositories por modulo.
- Estandarizar validaciones con **Form Requests** en todos los endpoints de escritura.

### Mediano plazo (1 a 2 meses)

- Incorporar **eventos y listeners** para procesos secundarios (notificaciones, auditoria, post-procesamiento).
- Aplicar **policies** por recurso para complementar gates por rol.
- Definir contrato de repositorio en modulos de mayor crecimiento (Players, Teams, Matches, Trainings).

### Largo plazo (3+ meses)

- Evaluar modularizacion interna por bounded contexts.
- Preparar estrategia para extraer servicios solo si hay necesidad real de desacoplar el monolito.